```
!git clone https://github.com/rishabhnmishra/Python_Diwali_Sales_Analysis.git
```

```
Cloning into 'Python_Diwali_Sales_Analysis'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 19 (delta 2), reused 0 (delta 0), pack-reused 13 (from 1)
Receiving objects: 100% (19/19), 666.76 KiB | 3.75 MiB/s, done.
Resolving deltas: 100% (4/4), done.
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv("/content/Python_Diwali_Sales_Analysis/Diwali Sales Data.csv",encoding='unicode_escape')
```

```
df.shape    #rows and columns
```

```
(11251, 15)
```

```
df.head(10)
```

	User_ID	Cust_name	Product_ID	Gender	Age Group	Age	Marital_Status	State	Zone	Occupatio
0	1002903	Sanskriti	P00125942	F	26-35	28	0	Maharashtra	Western	Healthcar
1	1000732	Kartik	P00110942	F	26-35	35	1	Andhra Pradesh	Southern	Gov
2	1001990	Bindu	P00118542	F	26-35	35	1	Uttar Pradesh	Central	Automobil
3	1001425	Sudevi	P00237842	M	0-17	16	0	Karnataka	Southern	Constructio
4	1000588	Joni	P00057942	M	26-35	28	1	Gujarat	Western	Foo Processin
5	1000588	Joni	P00057942	M	26-35	28	1	Himachal Pradesh	Northern	Foo Processin
6	1001132	Balk	P00018042	F	18-25	25	1	Uttar Pradesh	Central	Lawye
7	1002092	Shivangi	P00273442	F	55+	61	0	Maharashtra	Western	IT Sectc
8	1003224	Kushal	P00205642	M	26-35	35	0	Uttar Pradesh	Central	Gov
9	1003650	Ginny	P00031142	F	26-35	26	1	Andhra Pradesh	Southern	Medi

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	User_ID	Age	Marital_Status	Orders	Amount	Status	unnamed1
count	1.125100e+04	11251.000000	11251.000000	11251.000000	11239.000000	0.0	0.0
mean	1.003004e+06	35.421207	0.420318	2.489290	9453.610858	NaN	NaN
std	1.716125e+03	12.754122	0.493632	1.115047	5222.355869	NaN	NaN
min	1.000001e+06	12.000000	0.000000	1.000000	188.000000	NaN	NaN
25%	1.001492e+06	27.000000	0.000000	1.500000	5443.000000	NaN	NaN
50%	1.003065e+06	33.000000	0.000000	2.000000	8109.000000	NaN	NaN
75%	1.004430e+06	43.000000	1.000000	3.000000	12675.000000	NaN	NaN
max	1.006040e+06	92.000000	1.000000	4.000000	23952.000000	NaN	NaN

```
df.isnull().sum()        #finding the null values in a particular column
```

	0
User_ID	0
Cust_name	0
Product_ID	0
Gender	0
Age Group	0
Age	0
Marital_Status	0
State	0
Zone	0
Occupation	0
Product_Category	0
Orders	0
Amount	12
Status	11251
unnamed1	11251

dtype: int64

```
df.isnull().sum().sum()
```

```
np.int64(22514)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11251 entries, 0 to 11250
Data columns (total 15 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   User_ID           11251 non-null  int64  
 1   Cust_name         11251 non-null  object 
 2   Product_ID        11251 non-null  object 
 3   Gender            11251 non-null  object 
 4   Age Group         11251 non-null  object 
 5   Age               11251 non-null  int64  
 6   Marital_Status    11251 non-null  int64  
 7   State             11251 non-null  object 
 8   Zone              11251 non-null  object 
 9   Occupation        11251 non-null  object 
 10  Product_Category  11251 non-null  object 
 11  Orders            11251 non-null  int64  
 12  Amount            11239 non-null  float64
 13  Status            0 non-null      float64
 14  unnamed1          0 non-null      float64
dtypes: float64(3), int64(4), object(8)
memory usage: 1.3+ MB
```

```
df.drop(["Status","unnamed1"], axis=1, inplace=True)    #axis means vertical row delete inplace saves v
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipython-input-182/4178278816.py in <cell line: 0>()
----> 1 df.drop(["Status","unnamed1"], axis=1, inplace=True)    #axis means vertical row delete inplace
saves value.

                              3 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in drop(self, labels, errors)
   7068         if mask.any():
   7069             if errors != "ignore":
-> 7070                 raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7071             indexer = indexer[~mask]
   7072         return self.delete(indexer)

KeyError: "['Status', 'unnamed1'] not found in axis"
```

Next steps: Explain error

```
df.describe()
```

	User_ID	Age	Marital_Status	Orders	Amount
count	1.125100e+04	11251.000000	11251.000000	11251.000000	11239.000000
mean	1.003004e+06	35.421207	0.420318	2.489290	9453.610858
std	1.716125e+03	12.754122	0.493632	1.115047	5222.355869
min	1.000001e+06	12.000000	0.000000	1.000000	188.000000
25%	1.001492e+06	27.000000	0.000000	1.500000	5443.000000
50%	1.003065e+06	33.000000	0.000000	2.000000	8109.000000
75%	1.004430e+06	43.000000	1.000000	3.000000	12675.000000
max	1.006040e+06	92.000000	1.000000	4.000000	23952.000000

```
df.info()               #it was showing error because i ran the code twice after deleting the rows above
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11251 entries, 0 to 11250
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   User_ID           11251 non-null  int64
 1   Cust_name         11251 non-null  object
 2   Product_ID        11251 non-null  object
 3   Gender            11251 non-null  object
 4   Age Group         11251 non-null  object
 5   Age               11251 non-null  int64
 6   Marital_Status    11251 non-null  int64
 7   State             11251 non-null  object
 8   Zone              11251 non-null  object
 9   Occupation        11251 non-null  object
 10  Product_Category  11251 non-null  object
 11  Orders            11251 non-null  int64
 12  Amount            11239 non-null  float64
dtypes: float64(1), int64(4), object(8)
memory usage: 1.1+ MB
```

```
df.dropna(inplace=True)
```

```
df.info()  #rows delete
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 11239 entries, 0 to 11250
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   User_ID           11239 non-null  int64
 1   Cust_name         11239 non-null  object
```

```
 2   Product_ID        11239 non-null  object
 3   Gender            11239 non-null  object
 4   Age Group         11239 non-null  object
 5   Age               11239 non-null  int64
 6   Marital_Status    11239 non-null  int64
 7   State             11239 non-null  object
 8   Zone              11239 non-null  object
 9   Occupation        11239 non-null  object
 10  Product_Category  11239 non-null  object
 11  Orders            11239 non-null  int64
 12  Amount            11239 non-null  float64
dtypes: float64(1), int64(4), object(8)
memory usage: 1.2+ MB
```

data type change

```
df["Amount"]=df["Amount"].astype('int')
```

```
df["Amount"].dtypes
```

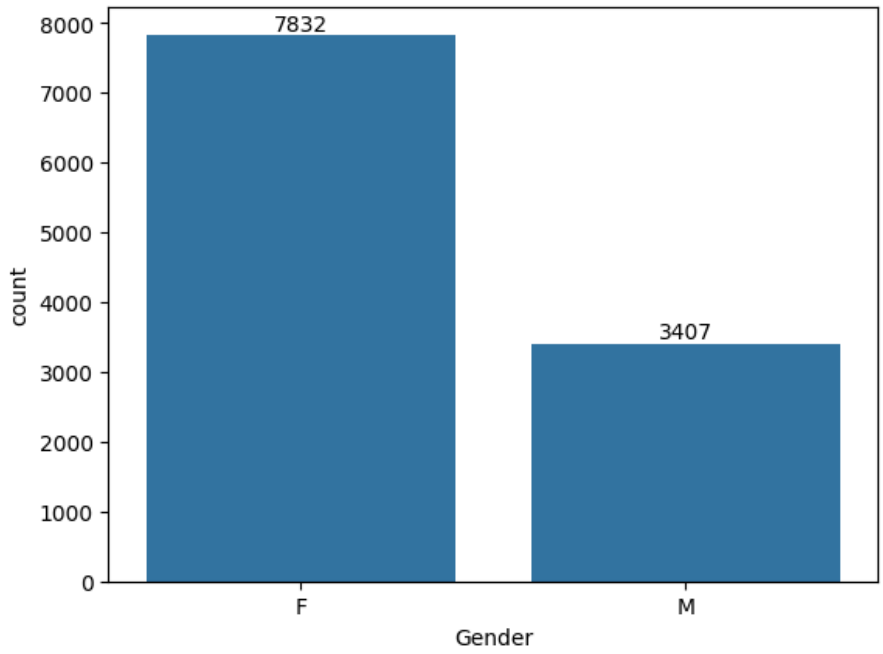
```
dtype('int64')
```

```
Start coding or generate with AI.
```

Exploratory data analysis

```
a=sns.countplot(x="Gender",data=df)
for container in a.containers:
    a.bar_label(container)

plt.show()
```

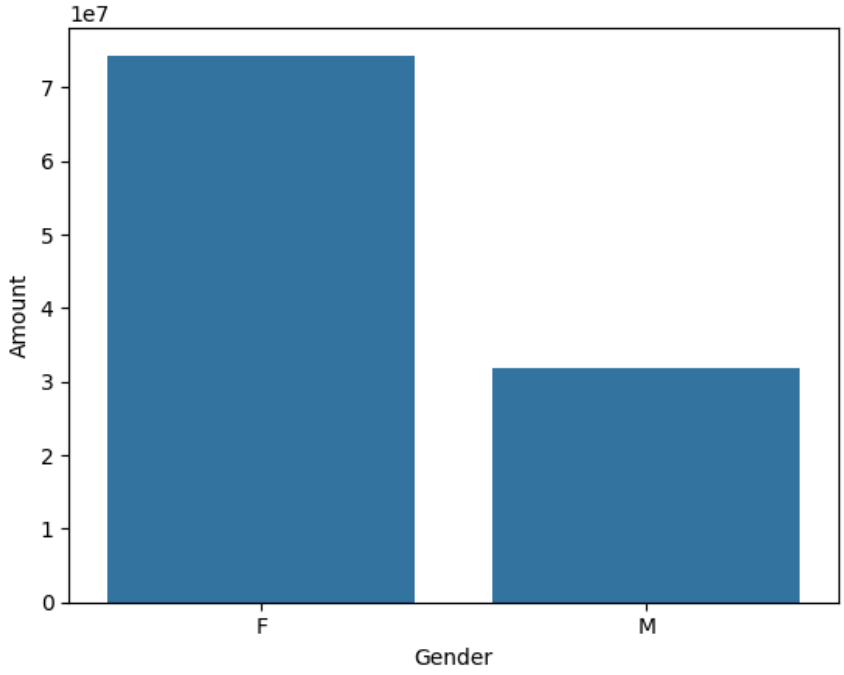



```
df.groupby(["Gender"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False)
```

	Gender	Amount
0	F	74335853
1	M	31913276

```
x=df.groupby(["Gender"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False)
sns.barplot(x="Gender",y="Amount",data=x)
```

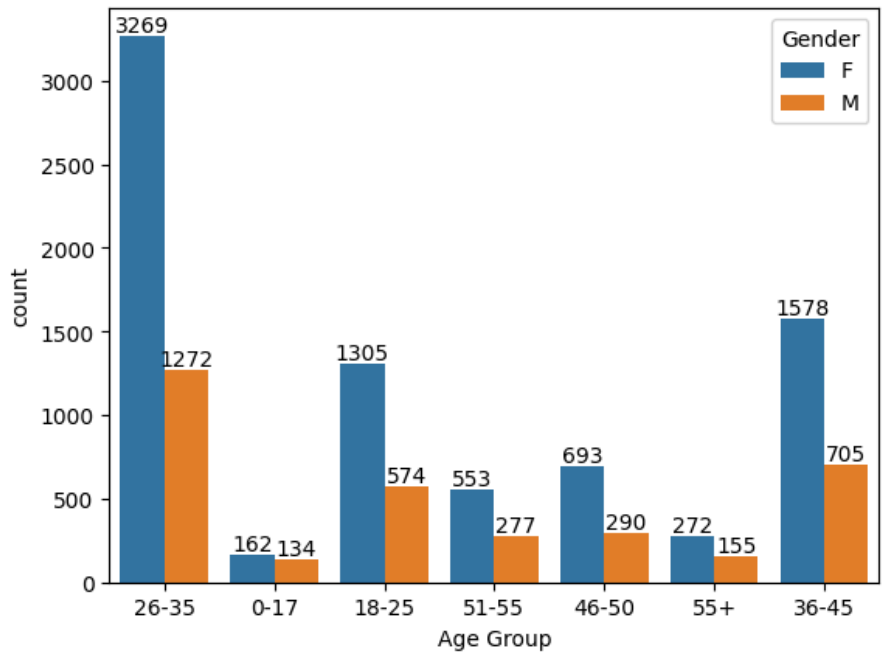
```
<Axes: xlabel='Gender', ylabel='Amount'>
```



> females are the majority buyers

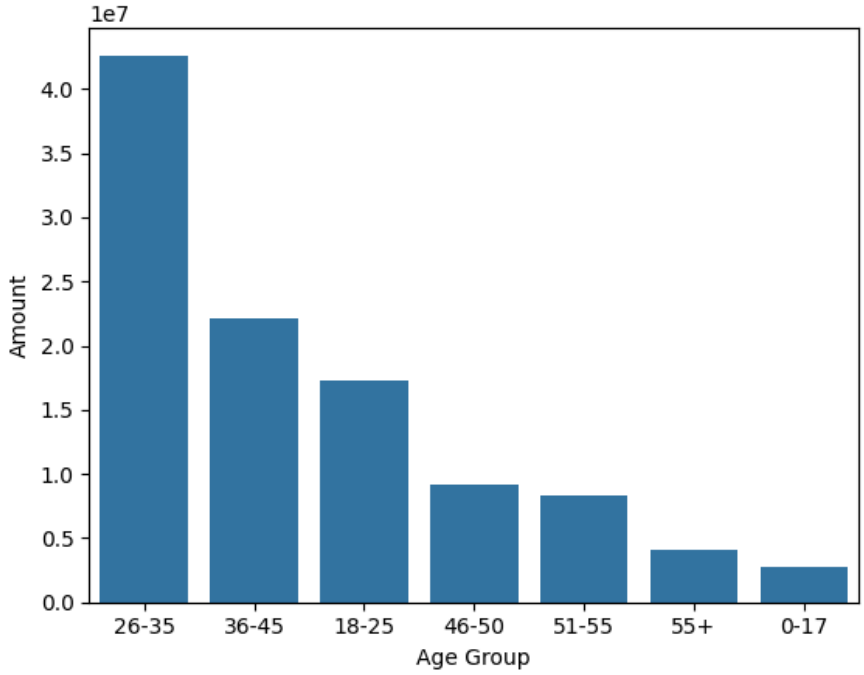
age group

```
a=sns.countplot(x="Age Group",data=df, hue="Gender")
for container in a.containers:
    a.bar_label(container)
```




```
x=df.groupby(["Age Group"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False)
sns.barplot(x="Age Group",y="Amount",data=x)
```

```
<Axes: xlabel='Age Group', ylabel='Amount'>
```



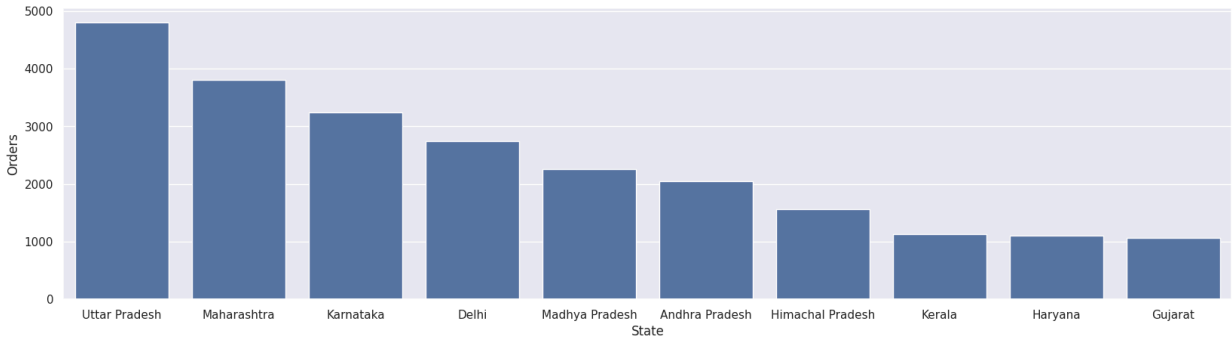
Start coding or generate with AI.

26-35 is the age group where most of the shopping happens!

Start coding or generate with AI.

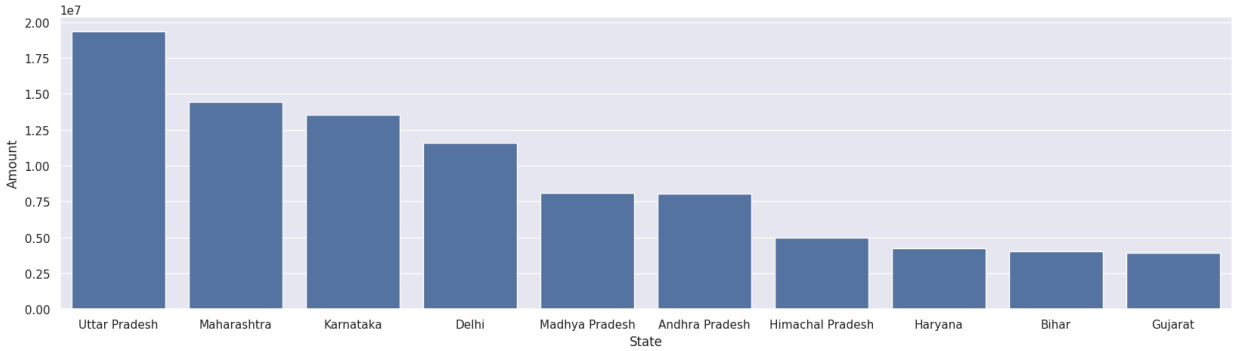
```
x=df.groupby(["State"],as_index=False)["Orders"].sum().sort_values(by="Orders",ascending=False).head(10
sns.set(rc={"figure.figsize":(20,5)})
sns.barplot(x="State",y="Orders",data=x)
```

```
<Axes: xlabel='State', ylabel='Orders'>
```

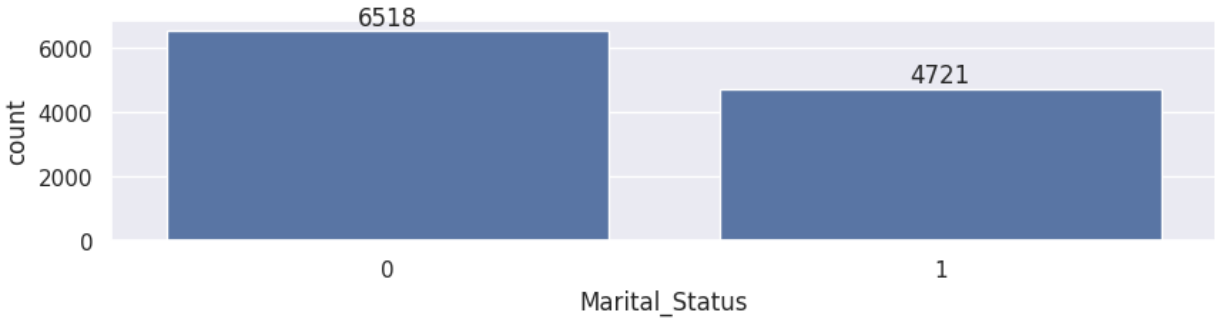



```
x=df.groupby(["State"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False).head(10
sns.set(rc={"figure.figsize":(20,5)})
sns.barplot(x="State",y="Amount",data=x)
```

```
<Axes: xlabel='State', ylabel='Amount'>
```

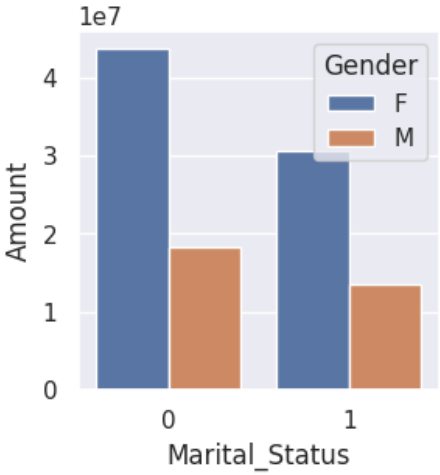



```
a=sns.countplot(x="Marital_Status",data=df,)
sns.set(rc={"figure.figsize":(5,2)})
for container in a.containers:
    a.bar_label(container)
```




```
x=df.groupby(["Marital_Status","Gender"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascend
sns.set(rc={"figure.figsize":(3,3)})
sns.barplot(x="Marital_Status",y="Amount",data=x, hue="Gender")
```

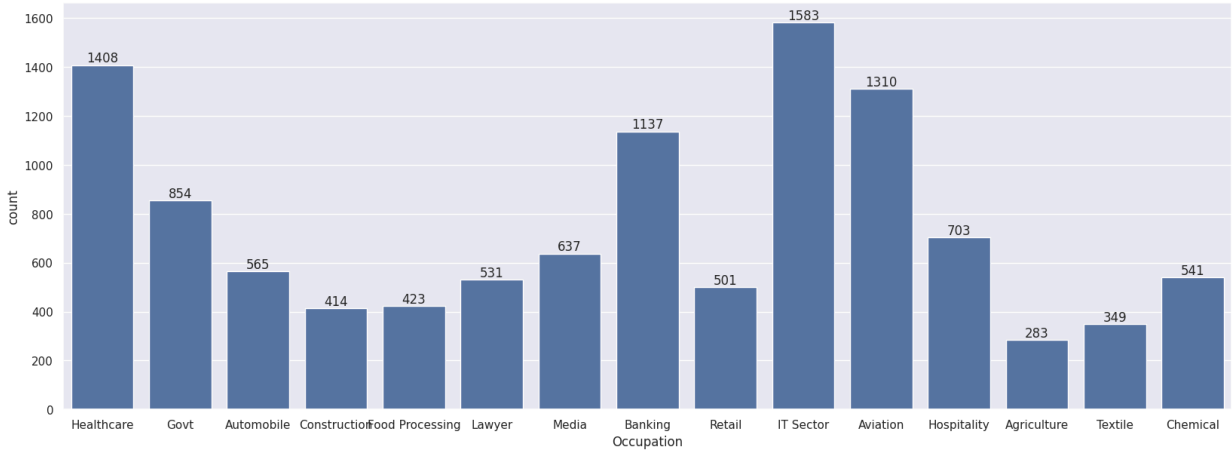
```
<Axes: xlabel='Marital_Status', ylabel='Amount'>
```



married women are the highest number of buyers.

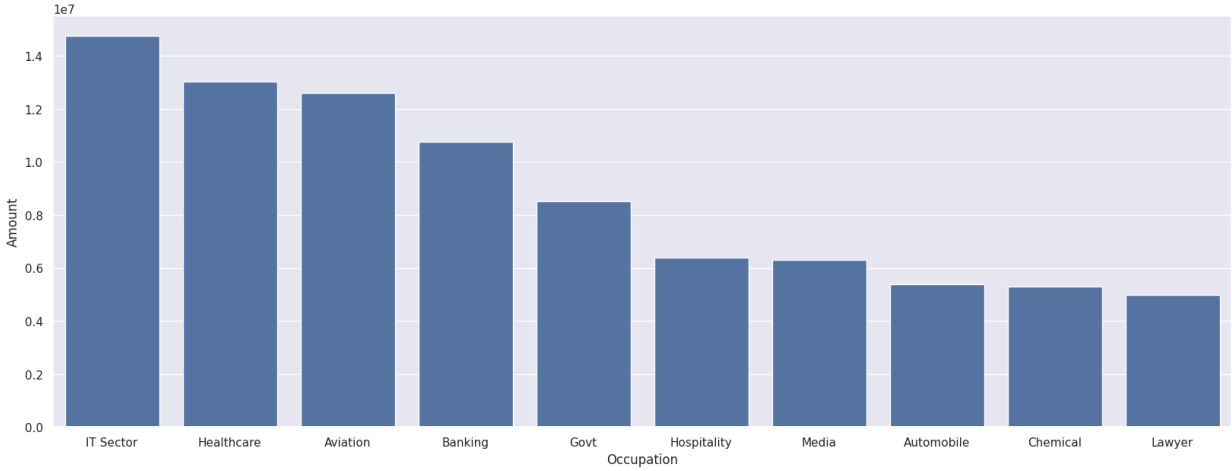
Occupation

```
a=sns.countplot(x="Occupation",data=df,)
sns.set(rc={"figure.figsize":(20,7)})
for container in a.containers:
    a.bar_label(container)
```



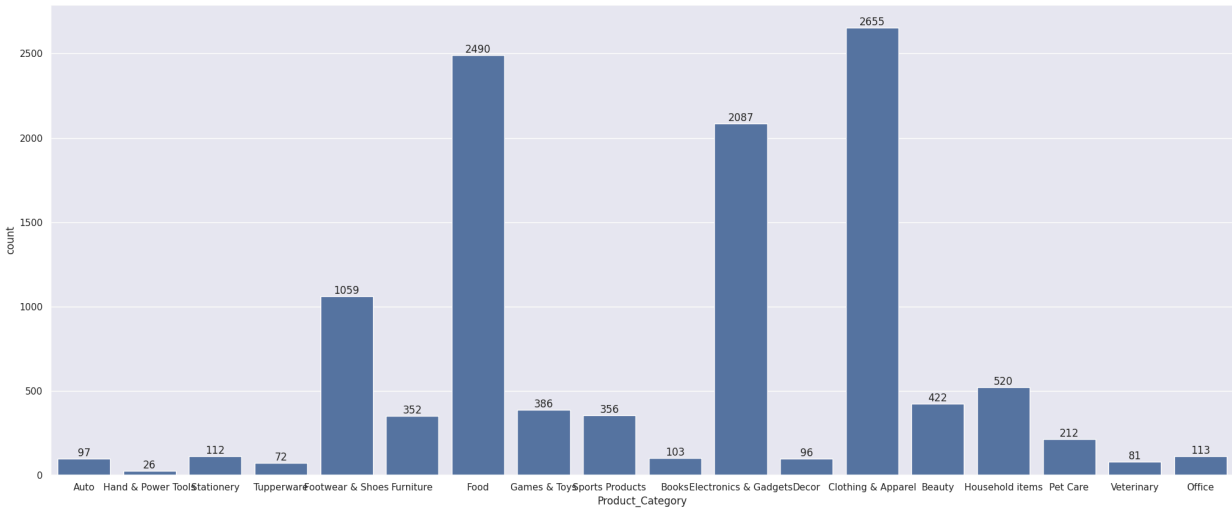

```
x=df.groupby(["Occupation"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False).he
sns.set(rc={"figure.figsize":(20,7)})
sns.barplot(x="Occupation",y="Amount",data=x, )
```

```
<Axes: xlabel='Occupation', ylabel='Amount'>
```



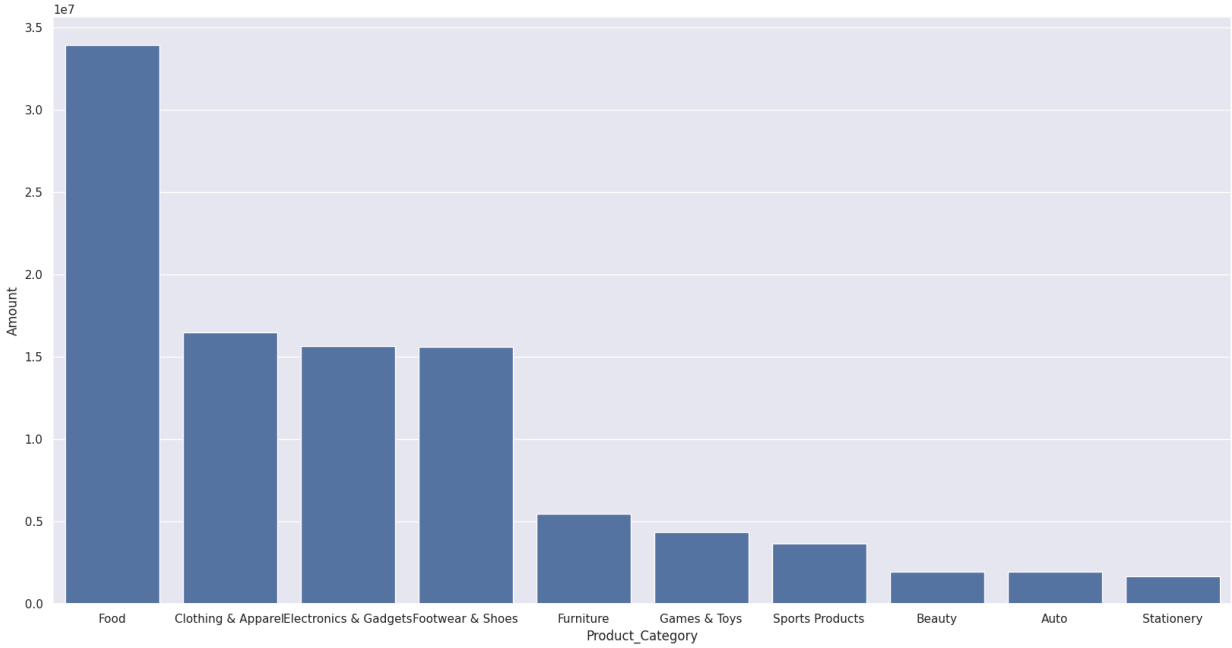
moat of the buyers are working in it healtcare and aviation

```
a=sns.countplot(x="Product_Category",data=df)
sns.set(rc={"figure.figsize":(25,10)})
for container in a.containers:
    a.bar_label(container)
```

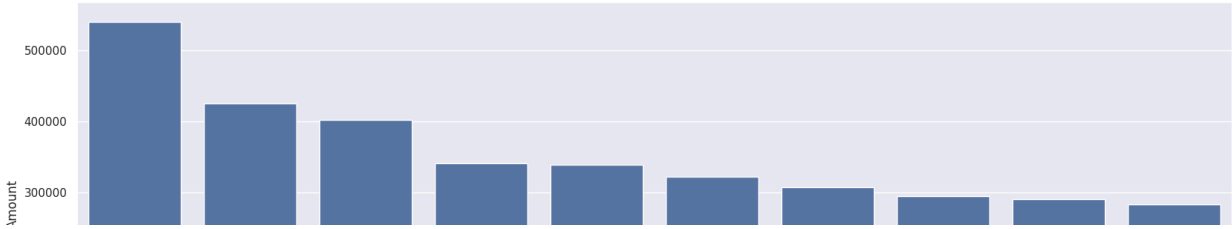


```
x=df.groupby(["Product_Category"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=Fal
sns.set(rc={"figure.figsize":(20,10)})
sns.barplot(x="Product_Category",y="Amount",data=x)
```

```
<Axes: xlabel='Product_Category', ylabel='Amount'>
```



food electronics and clothing are the most purchased products.

```
x=df.groupby(["Product_ID"],as_index=False)["Amount"].sum().sort_values(by="Amount",ascending=False).he
sns.set(rc={"figure.figsize":(20,7)})
sns.barplot(x="Product_ID",y="Amount",data=x)
```

```
<Axes: xlabel='Product_ID', ylabel='Amount'>
```